

```
1  (*<*)
2  theory Week_1
3  imports Main
4  begin
5  (*>*)
6  text <\ExerciseSheet{6}{}>
7
8  text <Important Note: Please download Isabelle from the link provided in the slides and bring a
9      laptop to the large group tutorial. Performing the proofs in Isabelle will be an integral part
10     of those tutorials.>
11
12
13  text <Before you start, please open a new Isabelle/HOL theory file called Week\_1.thy and add the
14      following to its beginning.
15
16  \noindent theory Week\_1\
17  imports Main\
18  begin
19  >
20
21  text<\Exercise{Addition Commutes}>
22
23
24  text<
25  Consider the function @{term "(+)"} defined in Isabelle/HOL for natural numbers.
26
27  Prove the following theorem, first pen-and-paper, and then formally in Isabelle.
28  >
29  (*
30  thm add_0_right
31
32  (*Note that 0 is also overloaded*)
33
34  thm add_0_left
35
36  thm add_Suc_right
37
38  thm add_Suc
39  *)
40  (*
41    Note that when we prove this lemma, we will have a lemma where there is a universal quantification
42    on n and m
43  *)
44  lemma add_commutes: "(n::nat) + m = m + n"
45
46  text <\paragraph{Hints:}
47
48  1. Perform the proof by inuction on <n>, using substitutions <subst>, and forward reasoning using
```

```

49   the method <rule>.
50
51   2. You will need to identify multiple lemmas about how <add> works. Do not prove those lemmas. Use
52     <sorry> as the proof. This is the top-down approach.
53
54   3. There is a proof which uses the following lemmas:
55
56     <add_0_right, add_0_left, add_Suc_right, add_Suc>
57   >
58   (*<*)
59   proof(induction m)
60     case 0
61     have 2: "n + 0 = n"
62       apply(subst (1) add_0_right)
63       ..
64     have 4: "0 + n = n"
65       apply(subst (1) add_0_left)
66       ..
67
68     have 5: "n + 0 = n"
69       using 2
70       .
71     from 5 show ?case
72       by(subst 4)
73   next
74     case (Suc m)
75
76     hence 2: "n + (Suc m) = Suc (n + m)"
77       apply(subst add_Suc_right)
78       ..
79
80     hence 4: "(Suc m) + n = Suc (m + n)"
81       apply(subst add_Suc)
82       ..
83
84     hence 6: "Suc (n + m) = Suc (m + n)"
85       (*Show IH*)
86       thm Suc.IH
87       (*Note that in order for the IH to apply here, we do not have to do any instantiations*)
88       apply(subst Suc.IH)
89       ..
90
91     hence 7: "n + (Suc m) = Suc (m + n)"
92       by(subst (asm) 2[symmetric])
93
94     thus ?case
95       by(subst (asm) 4[symmetric])
96
97   qed

```

```
98 |
99 | (*show them how to use then and hence*)
100 | (*Show them how the lemma is universally quantified*)
101 | (*Tell them it is an exercise to perform the proof with the equational style*)
102 | (*>*)
103 |
104 | text <\Exercise{Multiplication is Monotone}>
105 |
106 | text <Consider the multiplication function @{term "("} defined in Isabelle. Prove the following
107 |     property of it, i.e.\ that it is monotonically increasing in both arguments. You should prove
108 |     it both, pen-and-paper and in Isabelle.>
109 | (*
110 |
111 | thm refl
112 |
113 | thm diff_le_self
114 |
115 | thm mult_le_mono2
116 |
117 | thm mult_zero_right
118 |
119 | thm le0
120 |
121 | thm mult_Suc_right
122 |
123 | thm add_mono
124 |
125 | thm Suc_eq_plus1
126 |
127 | thm le_diff_conv
128 |
129 | *)
130 |
131 | lemma mult_mono:
132 |     fixes a b c d:: nat - <Note the alternative way of fixing the types of the constants>
133 |     assumes "a ≤ b" "c ≤ d"
134 |     shows "a * c ≤ b * d"
135 |
136 | text <
137 | Your pen-and-paper proof should indicate
138 |
139 | 1. what lemmas are you assuming,
140 |
141 | 2. on what variable are you performing induction,
142 |
143 | 3. what are assumptions and the proof goals in the base case and the step case, and what is the
144 |     induction hypothesis, and
145 |
146 | 4. how are you instantiating the induction hypothesis, i.e. how do you show that its assumptions
```

```

147     hold and which quantified variables are instantiated with which constants.
148
149 \paragraph{Hints:}
150 1. The proof should be by induction
151
152 2. If the induction hypothesis is too weak, you might need to strengthen by generalising/
153    universally quantifying over one of the variables. E.g. if you want to universally quantify on
154    a variable  $\langle x \rangle$ , then you can use  $\langle \text{induction } .. \text{arbitrary: } x \rangle$ .
155
156 3. The following lemmas suffice to prove the goal with only substitution, forward reasoning, and
157    induction:
158
159      $\langle \text{refl, diff\_le\_self, mult\_le\_mono2, mult\_zero\_right, le0, mult\_Suc\_right, add\_mono, Suc\_eq\_plus1,}$ 
160      $\text{le\_diff\_conv} \rangle$ 
161
162 4. When you have a term like  $\langle x - y \rangle$ , where  $\langle x \rangle$  is a  $\langle \text{nat} \rangle$ , in the goal or the assumptions, you
163    should perform proof by case analysis on whether  $\langle y \leq x \rangle$ . This is because of the way  $\langle - \rangle$  is
164    defined for natural numbers, where  $\langle x - y = 0 \rangle$ , for any  $\langle x \leq y \rangle$ .
165  >
166
167 (*Note that finding the proof is like solving a puzzle where all the pieces fit together*)
168
169 (*<*)
170 using assms
171 proof(induction d arbitrary: c)
172   case 0
173   then show ?case
174     by auto
175 next
176   case (Suc d)
177   from  $\langle c \leq \text{Suc } d \rangle$  have 1: " $c \leq d + 1$ "
178     apply (subst Suc_eq_plus1[symmetric])
179     .
180   hence " $c - 1 \leq d$ "
181     apply(subst le_diff_conv)
182     .
183
184   (*We can instantiate the I.H. with what we have, and then get the following: *)
185
186   hence 2: " $a * (c - 1) \leq b * d$ "
187     (*forward reasoning exampe*)
188     apply(rule Suc.IH[OF  $\langle a \leq b \rangle$ ])
189     .
190
191   (*However, we cannot directly with the available lemmas go from this to the goal.*/)
192   (*o show this, we have to consider two cases,  $c = 0$  and not *)
193   show ?case
194   proof(cases c)
195     case 0

```

```

196   have "a * c = a * 0"
197     apply(subst 0)
198     ..
199   also have 3: "... = 0"
200     apply(subst mult_zero_right)
201     ..
202   also have 4: "... ≤ b * (Suc d)"
203     using le0
204     .
205   finally show ?thesis
206     .
207 next
208   case (Suc nat)
209   have "(c - 1) = Suc nat - 1"
210     apply(subst Suc)
211     ..
212   also have "... = nat"
213     apply(subst diff_Suc_1)
214     ..
215   finally have "c - 1 = nat"
216     .
217
218   have 3: "a * nat ≤ b * d"
219     using 2
220     apply(subst <c - 1 = nat>[symmetric])
221     .
222
223   have "a * c = a * (Suc nat)"
224     apply(subst Suc)
225     ..
226   also have "... = a + a * nat"
227     apply(subst mult_Suc_right)
228     ..
229   finally have 4: "a * c = a + a * nat"
230     .
231
232   have 5: "b * Suc d = b + b * d"
233     apply (subst mult_Suc_right)
234     ..
235
236   have "a + a * nat ≤ b + b * d"
237     using add_mono[OF <a ≤ b> 3]
238     .
239
240   then have "a * c ≤ b + b * d"
241     apply(subst 4)
242     .
243   then show ?thesis
244     apply(subst 5)

```

```
245 |  
246   qed  
247 qed  
248 (*>*)  
249  
250  
251 (*<*)  
252 end  
253 (*>*)
```